EnduroSense: Uncertainty-Aware Mission Feasibility Prediction for UAVs

A Project Report submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Aditya Harikrishnan (MIS No.)

Under the Supervision of: [Supervisor's Name]

Semester: [Semester]



Department of Computer Science and Engineering

Indian Institute of Information Technology, Pune

(An Institute of National Importance by an Act of Parliament)

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"EnduroSense: Uncertainty-Aware Mission Feasibility Prediction for UAVs"** submitted by **Aditya Harikrishnan** bearing the **MIS No: [MIS No.]**, in completion of the project work under the guidance of **[Supervisor's Name]** is accepted for the project report submission in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in the **Department of Computer Science and Engineering**, Indian Institute of Information Technology, Pune (IIIT Pune), during the academic year **2026-27**.

Supervisor's Name	**HoD Name**
Internship-I Supervisor	Head of the Department
Designation of the Supervisor	Assistant Professor
Department of the Supervisor	Department of CSE
IIIT Pune	IIIT Pune

Internship Project Viva-voce held on _____

Undertaking for Plagiarism

I **Aditya Harikrishnan** solemnly declare that the research work presented in the **report** titled **"EnduroSense: Uncertainty-Aware Mission Feasibility Prediction for UAVs"** is solely **my** research work with no significant contribution from any other person. Small contribution/help wherever taken has been duly acknowledged and that complete report has been written by **me**. I understand the zero tolerance policy of **Indian Institute of Information Technology, Pune** towards plagiarism. Therefore **I** declare that no portion of my **report** has been plagiarized and any material used as reference is properly referred/cited. I undertake that if I am found guilty of any formal plagiarism in the above titled thesis even after award of the degree, the Institute reserves the rights to withdraw/revoke my **B.Tech** degree.

Student's Name and Signature with Date: Aditya Harikrishnan

Conflict of Interest

Project title: EnduroSense: Uncertainty-Aware Mission Feasibility Prediction for UAVs

The author whose name is listed immediately below certifies that the author has NO affiliations with or involvement in any organization or entity with any financial interest (such as honoraria; educational grants; participation in speakers' bureaus; membership, employment, consultancies, stock ownership, or other equity interest; and expert testimony or patent-licensing arrangements), or non-financial interest (such as personal or professional relationships, affiliations, knowledge or beliefs) in the subject matter or materials discussed in this manuscript.

Student's Name and Signature with Date: Aditya Harikrishnan

ACKNOWLEDGEMENT

This project would not have been possible without the help and cooperation of many. I would like to thank the people who helped me directly and indirectly in the completion of this project work.

First and foremost, I would like to express my gratitude to our honorable Director, **Prof. Vineet Kansal**, for providing his kind support in various aspects. I would like to express my gratitude to my project guide **[Supervisor's Name]**, **Department of CSE**, for providing excellent guidance, encouragement, inspiration, constant and timely support throughout this **project**. I would like to express my gratitude to the **Head of Department [HoD Name]**, **Department of CSE**, for the kind support extended in various aspects. I would also like to thank all the faculty members in the **Department of CSE** and my classmates for their steadfast and strong support and engagement with this project.

I am also grateful to the researchers at Carnegie Mellon University who collected and openly published the package-delivery quadcopter dataset on which this work is built. Without real flight telemetry of this quality, the evaluation reported here would not have been possible.

Aditya Harikrishnan

Abstract

Battery-powered multirotor unmanned aerial vehicles (UAVs) used for package delivery must decide, before every sortie, whether the battery on board can complete the mission and still land with a safe reserve. The conventional answer, an estimate of the remaining flight time compared with the planned mission duration, is unreliable: remaining minutes are computed from the power drawn at the moment of the estimate rather than the power the next mission will demand, and a single number says nothing about how far it can be trusted.

This project presents EnduroSense, an uncertainty-aware framework that reformulates the go/no-go question in terms of energy. Model A estimates the energy a battery can still deliver above a 22.6 V rest-voltage reserve from its telemetry, using an ensemble of five gated recurrent unit (GRU) networks selected after a battery-grouped comparison of Linear Regression, Random Forest, XGBoost, LSTM and GRU models against non-learning baselines. Model B estimates the energy a planned mission requires by composing physics-informed models of climb, cruise, hover and descent, corrected by gradient boosting. Both predictions receive calibrated ranges through conformal calibration grouped by battery, and are combined into a probability of mission success, P(success). A mission is approved only if $P(\text{success}) \geq 0.95$.

The framework was developed on 169 flights of a DJI Matrice 100 from the public Carnegie Mellon University package-delivery dataset and evaluated once on a locked test set of 40 flights from 16 unseen battery chains, against success criteria fixed in advance. On the test set Model A achieved a mean absolute error of 2.19 Wh, against 2.90 Wh for the best non-learning baseline, and Model B predicted mission energy within 3.1% (5.7% on routes never seen in training). The conventional minutes-left rule approved 12.4% of the missions that would have breached the reserve; EnduroSense approved 0.61%, and none at take-off decisions, at the cost of refusing 24% of feasible but mostly marginal missions. Two simulated extensions, an energy-aware fleet planner and an energy-aware choice of where to process on-board data, met all of their pre-registered criteria.

Keywords: Unmanned aerial vehicles, Battery energy estimation, Mission energy prediction, Uncertainty quantification, Conformal prediction, Deep learning, Fleet scheduling

TABLE OF CONTENTS

Abstract i

List of Figures / Symbols / Nomenclature iii

List of Tables iv

1 Introduction 1

1.1 Overview of Work 1
1.2 Literature Review 2
1.3 Motivation of the Work 3
1.4 Research Gap 4

2 Problem Statement 5

2.1 Research Objectives 5
2.2 Analysis and Design 5

3 Proposed Work 8

3.1 Methodology of Work 8
3.1.1 Data preparation and flight-phase segmentation 8
3.1.2 Battery-chain reconstruction and the locked split 8
3.1.3 Model A: energy available 9
3.1.4 Model B: energy required 10
3.1.5 Calibrated uncertainty 11
3.1.6 Feasibility decision and evaluation design 12
3.1.7 Extensions: fleet planner and edge computing 12
3.2 Hardware & Software Specifications 12
3.3 Dataset Description 13

4 Results and Discussion 15

4.1 Evaluation Protocol 15
4.2 Energy Available (Model A) 15
4.3 Energy Required (Model B) 16
4.4 Calibration of the Predictive Ranges 17
4.5 Feasibility Decisions 17
4.6 Fleet Simulation 18
4.7 Extensions: Fleet Planner and Edge Computing 19
4.8 Discussion and Limitations 20

5 Conclusion and Future Scope 21

References 22

List of Figures

Fig. 1: Overall architecture of EnduroSense 6
Fig. 2: Battery-grouped split and cross-validation 9
Fig. 3: Architecture of the Model A network 10
Fig. 4: Decomposition of a mission into phases 11
Fig. 5: Model A cross-validation error against test error 16
Fig. 6: Calibration of both models on test data 17
Fig. 7: Trade-off between unsafe approvals and wasted refusals 18
Fig. 8: Fleet simulation on unseen batteries 19
Fig. 9: The extensions on unseen batteries 20

Symbols and Nomenclature

E_{avail}	Energy available above the reserve (Wh)
E_{req}	Energy required by a mission (Wh)
E_{res}	Energy drawn before the rest voltage reaches the reserve (Wh)
V, I, P	Battery voltage (V), current (A) and power (W)
W, ρ, A	Vehicle weight (N), air density (kg/m³), rotor disc area (m²)
v, v_i, v_h	Airspeed, induced velocity and hover induced velocity (m/s)
$\hat{E}$, $\hat{\sigma}$	Ensemble estimate and spread of Model A (Wh)
τ	Approval threshold on P(success), 0.95
CV	Cross-validation (grouped by battery chain)
GRU / LSTM	Gated recurrent unit / long short-term memory network
MAE / MAPE	Mean absolute error / mean absolute percentage error
UAV	Unmanned aerial vehicle

List of Tables

Table 1: Summary of the reviewed literature 3

Table 2: Main software modules 7

Table 3: Input features of Model A 9

Table 4: Hardware and software environment 13

Table 5: Characteristics of the dataset 13

Table 6: Success criteria and outcome 15

Table 7: Model A error in cross-validation and on test 15

Table 8: Model B mission-energy error 16

Table 9: Decision quality of each rule 17

Table 10: Fleet simulation on unseen batteries 18

Table 11: Energy-aware fleet planner 19

Table 12: Choice of where to process data 19

Chapter 1
Introduction

1.1 Overview of Work

Small multirotor UAVs are increasingly used for last-mile package delivery, inspection and emergency logistics. Their endurance is limited by a lithium-polymer battery that typically supports only a few tens of minutes of flight, and the consequences of misjudging it are severe: a drone that drains its battery below a safe reserve risks a forced landing, damage to the airframe and payload, and harm to people on the ground. Every fleet operator therefore faces the same recurring decision before each sortie: *can this battery, in its present state, complete this particular mission and return with a safe reserve?*

The usual engineering answer is to estimate the remaining flight time of the battery and to approve a mission if the remaining minutes exceed the planned mission duration. The original brief of this project, "ML-Based UAV Battery / Remaining Flight-Time Prediction", followed that formulation. During the analysis it became clear that minutes are a weak basis for a go/no-go decision, because the same battery lasts very different times depending on what it is asked to do next, and because a single point estimate hides how uncertain it is.

EnduroSense extends the brief by reformulating the decision in terms of energy, which is additive across the parts of a mission, and by attaching a calibrated range to every prediction. It consists of three layers:

- **Model A (energy available):** the watt-hours the battery can still deliver before reaching a 22.6 V rest-voltage reserve, estimated from voltage and current telemetry by an ensemble of recurrent neural networks.
- **Model B (energy required):** the watt-hours a planned mission will consume, assembled from physics-informed models of climb, cruise legs, hover and descent that take only quantities known at planning time.
- **Feasibility layer:** the two calibrated predictive distributions are combined into P(success), the probability that the energy available covers the energy required, and a mission is approved only if P(success) $\geq \tau$ with $\tau = 0.95$. A fleet scheduler uses this rule to assign batteries to tasks.

The framework was built and validated on real flight telemetry: 209 flights of a DJI Matrice 100 quadcopter from the public Carnegie Mellon University (CMU) package-delivery dataset [1]. Development used 169 flights; 40 flights from 16 battery chains were locked away before any modelling and opened only once, for a final evaluation against criteria written down in advance. The main contributions of the work are:

1. A reconstruction of physical battery identity ("battery chains") from rest voltages, used as the unit of every train/test split so that no battery contributes to both training and evaluation.

2. A five-algorithm comparison for battery energy estimation under battery-grouped cross-validation, and a calibrated GRU ensemble that reduces error on unseen batteries from 2.90 Wh (best non-learning baseline) to 2.19 Wh.
3. A compositional, physics-first mission energy model that predicts unseen flights to 3.1% and, unlike pure tree models, extrapolates sensibly to unseen altitudes and payloads.
4. Battery-grouped conformal calibration of both models and a deterministic P(success) computation.
5. An evaluation of decision rules on real battery-state and mission pairs, in which unsafe approvals fall from 12.4% (minutes-left rule) to 0.61%, together with a fleet simulation and two simulated extensions requested by the guide.

The remainder of this report is organised as follows. Chapter 2 states the problem formally and lists the research objectives and the system design. Chapter 3 describes the methodology, the computing environment and the dataset. Chapter 4 reports the results on the locked test set, including the criteria that were not met, and Chapter 5 concludes with the future scope of the work.

1.2 Literature Review

Energy consumption of delivery drones. Zhang et al. [2] compared the energy models used in the drone delivery literature and showed that, for the same vehicle and mission, published models disagree by large factors, which motivates validating any energy model against measured flights rather than trusting its form. Abeywickrama et al. [3] built an empirical energy model for a small quadcopter from battery measurements across flight manoeuvres, showing that hover, climb and forward flight have markedly different power demands. Rodrigues et al. [1] released the dataset used in this project, a controlled grid of 209 delivery flights recording battery voltage, current, wind and position, and Rodrigues et al. [4] used the same data to build a first-principles energy model of take-off, cruise and landing for life-cycle emissions analysis. The physical basis for rotor power is momentum theory [5]: induced power falls with forward speed while parasitic power rises roughly with its cube, producing the U-shaped power-speed curve observed in the data.

Battery state and remaining-energy estimation. Battery management systems estimate state of charge from open-circuit voltage, coulomb counting and equivalent-circuit models [6]. For electric UAVs, Saha et al. [7] proposed a battery health management system that predicts end of discharge under an assumed future load. Such remaining-flight-time approaches depend critically on the assumed future power profile, which is exactly the quantity that changes from one mission to the next.

Learning algorithms. The brief required a comparison of five algorithm families: regularised linear regression, Random Forests [8], gradient-boosted trees in the form of XGBoost [9], and the recurrent architectures LSTM [10] and GRU [11], which read a time series in order and maintain an internal memory of it.

Uncertainty quantification. Quantile regression [12] and deep ensembles [13] produce predictive ranges, but neither guarantees that a nominal 90% range covers the truth 90% of the time on new data. Conformal prediction [14], [15] and conformalised quantile regression [16] calibrate ranges using held-out errors, with a coverage guarantee under exchangeability. Angelopoulos and Bates [17] discuss the care needed when data are grouped, as readings from one battery are.

Fleet operation and edge computing. Dorling et al. [18] formulated vehicle routing for drone delivery with an energy model that depends on payload and battery weight. Assignment of tasks to vehicles is classically solved with the Hungarian method [19]. Mobile edge computing [20] studies when a mobile device should offload computation to a nearby server, trading communication energy and delay against local computation. Table 1 summarises the works most relevant to this project.

Work	Focus	Method	Relevance and limitation
Rodrigues et al. [1]	Delivery-drone flight dataset	209 controlled flights, 8 Hz telemetry	Data source; no battery identity recorded
Rodrigues et al. [4]	Drone delivery energy	First-principles energy model	Closest mission-energy work; no calibrated uncertainty
Zhang et al. [2]	Survey of energy models	Comparison of published models	Models disagree widely; must be validated on data
Abeywickrama et al. [3]	Quadcopter energy	Empirical battery measurements	Phase-dependent power; single platform
Saha et al. [7]	UAV battery health	Model-based end-of-discharge prediction	Assumes a known future load profile
Lakshminarayanan et al. [13]	Predictive uncertainty	Deep ensembles	Used for Model A; not calibrated by itself
Romano et al. [16]	Interval calibration	Conformalised quantile regression	Basis of calibration; needs grouped data handling
Dorling et al. [18]	Drone delivery routing	Energy-aware vehicle routing	Deterministic energy; no risk of failure
Mao et al. [20]	Mobile edge computing	Offloading survey	Basis of the edge-computing extension

Table 1: Summary of the reviewed literature and its relevance to EnduroSense

1.3 Motivation of the Work

Remaining-flight-time estimates fail as a decision variable for two concrete reasons. First, remaining minutes are computed by dividing the remaining energy by the power being drawn now. A drone that is hovering or descending draws less power than the cruise legs ahead of it, so its remaining minutes are overstated precisely when the decision is taken. Second, a single number conveys no information about its reliability: two batteries may show the same estimate while one of them is far less certain, and a safe decision needs to know which.

The costs of the two possible errors are also highly asymmetric. Refusing a mission that would have succeeded wastes a sortie or a battery swap; approving a mission that cuts into the reserve risks the vehicle and its surroundings. A decision rule should therefore control the rate

of unsafe approvals explicitly, which requires calibrated uncertainty rather than a point estimate. Early experiments on development data confirmed the scale of the problem: the minutes-left rule approved 10.5% of the missions that would have breached the reserve.

1.4 Research Gap

The works reviewed leave the following gaps, which this project addresses:

- Remaining-flight-time methods assume a future load profile instead of predicting the energy of the specific mission that is about to be flown.
- Mission energy models are rarely tested on held-out batteries, days and routes, so their accuracy outside the conditions they were fitted on is largely unknown.
- Predictive ranges, where reported, are seldom shown to be calibrated on new data, and calibration with grouped, strongly dependent readings (thousands of samples from one battery) is not addressed.
- Random row-level train/test splits leak information between readings of the same battery and make models look better than they are.
- No work among those reviewed combines a battery energy prediction with a mission energy prediction into a calibrated probability of success and evaluates the resulting decisions against real outcomes and within a fleet.

Chapter 2
Problem Statement

Given the telemetry history of a battery up to the present time t and the plan M of a candidate mission (route legs, commanded speed, payload, altitude and expected wind), decide whether to approve the mission such that the rate of unsafe approvals, approvals of missions that would end below the reserve, is small, while as few feasible missions as possible are refused. The reserve is defined as a rest voltage of 22.6 V (about 3.77 V per cell for the six-cell pack). A mission succeeds if the energy available above the reserve at time t is at least the energy the mission requires, and EnduroSense approves it when

$$\text{approve} \Leftrightarrow P(\text{success}) \geq \tau, \tau = 0.95 \quad (1)$$

The reserve voltage and the threshold τ are single configuration settings, kept at the values above as the project defaults.

2.1 Research Objectives

1. **O1.** Reconstruct which flights shared a physical battery and build a train/test protocol in which no battery appears on both sides.
2. **O2.** Estimate the energy available above the reserve from battery telemetry, comparing the five algorithm families of the brief with non-learning baselines.
3. **O3.** Estimate the energy a planned mission requires from planning-time information only, including for routes and conditions not seen in training.
4. **O4.** Attach calibrated predictive ranges to both estimates and combine them into P(success).
5. **O5.** Evaluate decision rules, and a fleet that uses them, on real battery states and real recorded missions.
6. **O6.** Extend the system, as requested by the guide, with an energy-aware fleet planner and an energy-aware choice of where to process on-board data.

Four success criteria were fixed before the test set was opened: (C1) Model A beats both non-learning baselines; (C2) Model B is within 5% mean absolute percentage error, including on unseen routes; (C3) nominal 90% ranges cover between 88% and 92% of test cases; and (C4) EnduroSense makes better decisions than both the minutes-left rule and energy point estimates.

2.2 Analysis and Design

Figure 1 shows the overall architecture. The data layer cleans the raw telemetry, segments every flight into phases, integrates energy, reconstructs battery chains and produces a locked split. The prediction layer contains the two models, each followed by a calibration step. The decision layer turns the two predictive distributions into P(success), applies the threshold, and feeds a fleet scheduler and an interactive dashboard.

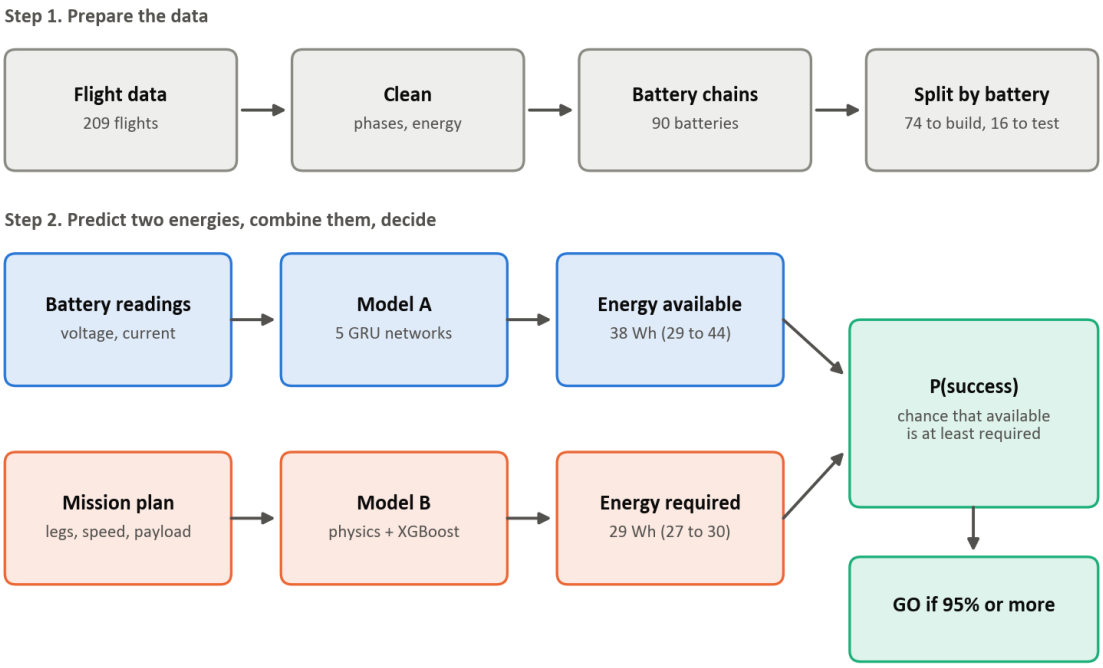


Fig. 1: Overall architecture of EnduroSense: data preparation, the two energy models and the feasibility decision

Several design decisions follow from the analysis in Chapter 1:

- **Energy instead of time.** Energy is additive over mission phases and independent of the momentary power draw, so the two models can be built and validated separately.
- **The battery chain as the unit of evaluation.** Readings from one battery share a common error, so every split, cross-validation fold and calibration weight is defined per battery.
- **A locked test set and pre-registered criteria.** The test chains were selected before modelling, gated in code and opened once, so that the reported results are not optimistic.
- **Physics-first mission modelling.** Physically structured components are preferred unless a pure learning model is clearly better, because missions outside the training grid must still be predicted sensibly.
- **Abstention.** Model A declines to answer when no pre-flight rest voltage exists, and the decision is then no-go.

The implementation is a Python package with one module per responsibility, a numbered pipeline of scripts and an automated test suite. Table 2 lists the main modules.

Module	Responsibility
data/	Loading, cleaning, flight phases, energy integration, wind, battery chains, locked split
features/	Model A labels and causal features; Model B leg-level and flight-level targets
models/	Baselines, tabular (Linear, Random Forest, XGBoost) and sequence (LSTM, GRU) predictors; Model B components
uncertainty/	Battery-grouped conformal calibration and predictive distributions
feasibility.py	P(success) and the go/no-go rule
whatif.py, scheduler.py	Decision evaluation on real state and mission pairs; fleet simulation
fleet_planner.py, edge.py	Extensions: energy-aware fleet planner and edge-computing choice
access.py	Gate and log for every opening of the locked test set
app/	Streamlit dashboard that loads saved models and results

Table 2: Main software modules of the EnduroSense implementation

Chapter 3

Proposed Work

This chapter describes the methodology in the order of the pipeline: data preparation, battery chains and the locked split, the two energy models, their calibration, the decision rule with its evaluation design, and the two extensions. It then lists the computing environment and describes the dataset.

3.1 Methodology of Work

3.1.1 Data preparation and flight-phase segmentation

The raw telemetry was checked for missing values, time reversals, values outside physical limits and recording gaps (10 gaps longer than 1 s, the largest 5.3 s). Seventy-six isolated sensor glitches (0.03% of readings) were replaced by the median of their five neighbours and flagged. Every reading was labelled as ground (below 1.5 m or current under 2 A), cruise (horizontal speed at least 1.5 m/s), climb or descent (vertical speed above 0.5 m/s in magnitude) or hover, and fragments shorter than 1 s were merged into their neighbours; 195 of the 196 route flights follow the expected ground, climb, cruise, descent, ground sequence. Energy drawn is integrated reading by reading:

$$E_{\text{drawn}}(t) = \sum_{k:t_k \le t} V_k \, I_k \Delta t_k \tag{2}$$

where V_k and I_k are the battery voltage and current and Δt_k is the sampling interval. Ambient wind is estimated from the on-board anemometer at moments when the drone is airborne above 10 m and not moving sideways.

3.1.2 Battery-chain reconstruction and the locked split

The dataset does not record which physical battery was used on each flight. A battery at rest shows its charge as a voltage, and between two flights on the same battery the rest voltage can only stay level or recover slightly. Consecutive flights on the same day were therefore linked when the rest voltage changed by between −0.10 V and +0.40 V. This produced 90 battery chains, 71 of which contain more than one flight; alternative reasonable windows give between 87 and 92 chains.

Sixteen chains (40 flights) were set aside as the test set before any model was built. They were chosen from 5,000 random candidate splits as the one whose test part best matched the whole dataset in flight share, speed, payload and altitude mix, and the share of chains reaching the reserve. The longest route, R6, was forced into the test set so that the final evaluation includes a distance never seen in development. The split file is protected by a hash, the code refuses to load test rows unless the final evaluation requests them, and every opening is logged. The remaining 74 chains (169 flights) form the development data, divided into five folds by chain (Figure 2).

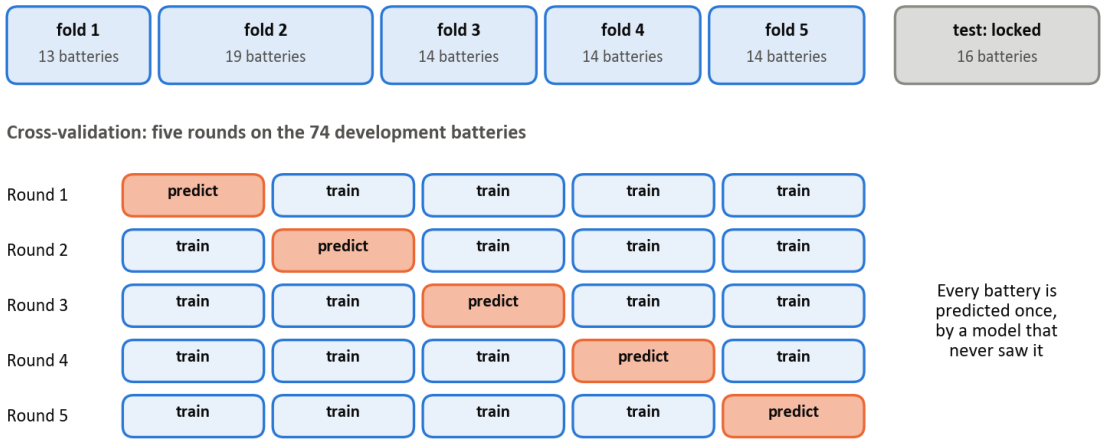


Fig. 2: Battery-grouped split: five development folds for cross-validation and a locked test set of 16 batteries

3.1.3 Model A: energy available

For a chain that was flown down to the reserve, the energy E_{res} drawn before its rest voltage reached 22.6 V is known, and the label at any time t is

$$E_{\text{avail}}(t) = E_{\text{res}} - E_{\text{drawn}}(t) \quad (3)$$

Because rest voltages are only observed between flights, the crossing point is located along a discharge curve fitted on development batteries (bias −0.04 Wh, spread 1.5 Wh); a straight-line interpolation was biased by +3.3 Wh. Chains that stop within 0.3 V of the reserve are extended along the same curve. Sixty of the 90 chains can be labelled, giving 29,592 one-second rows. The label itself is therefore uncertain by about 1.5 to 2.2 Wh, which bounds the accuracy any model can demonstrate. Model A uses 25 causal features, summarised in Table 3; a test perturbs every reading after time t and checks that no feature at or before t changes.

Group	Features
Instantaneous	Voltage, current, power
Rolling (10 s and 30 s)	Mean, spread and trend of voltage and current; mean power
Battery health	Internal-resistance estimate over 30 s; voltage sag below the rest voltage
Charge at insertion	Rest voltage before this flight; rest voltage when the battery was first used
Energy counting	Energy drawn in this flight and on this battery; time since first use
Context	Flight index on the battery; motors on or off
Sequence (LSTM/GRU)	Last 60 s of voltage, current, power, energy drawn and motor state at 2 Hz (120 steps)

Table 3: Input features of Model A (all computed from past readings only)

Three non-learning baselines set the bar: fixed capacity (every battery gives the same total), energy counting from the rest voltage at insertion, and a voltage lookup that re-reads the charge from the latest pre-flight rest voltage and subtracts the energy drawn in the current flight. The five algorithms were tuned by random search scored with battery-grouped cross-validation: ridge regression ($\lambda = 10$), Random Forest (depth 12, at least 20 rows per leaf),

XGBoost (400 trees of depth 3, learning rate 0.02), and two-layer LSTM and GRU networks with 64 units. The recurrent models read the 120-step sequence, and their final hidden state is concatenated with the 25 features and passed to a small output network (Figure 3). Networks were trained with the Huber loss [21] and early stopping on 15% of the training batteries.

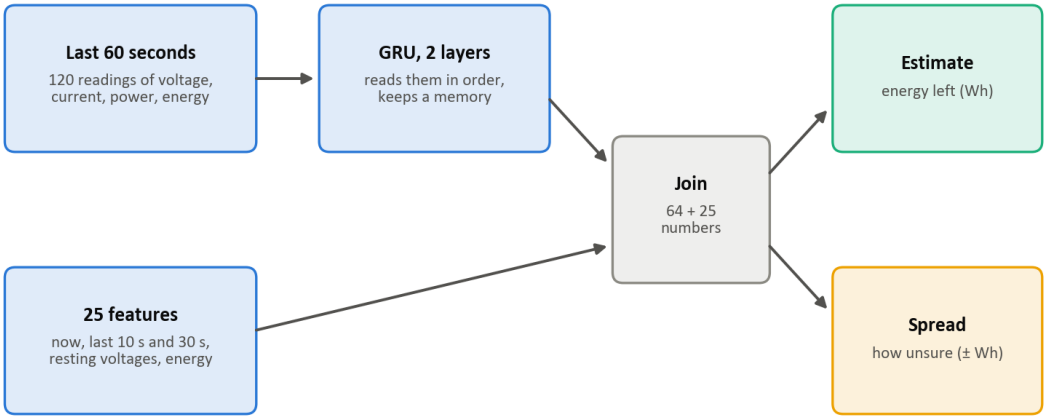


Fig. 3: Architecture of the Model A network: a two-layer GRU over the last minute of telemetry joined with 25 features, with estimate and spread heads

The final Model A is a deep ensemble [13] of five GRUs, each trained from a different random initialisation and early-stopping subset and each predicting an estimate and a spread. The spread head is trained after the estimate head is frozen, because joint training worsened the estimate (2.51 against 2.39 Wh). When no pre-flight rest voltage exists, errors are 10 to 20 Wh, so the model abstains and the decision is no-go.

3.1.4 Model B: energy required

A mission is decomposed into its phases (Figure 4) and its energy is the sum of their energies:

$$E_{\text{req}} = E_{\text{climb}} + \sum_{j=1}^{L} P_j\, t_j + E_{\text{hover}} + E_{\text{descent}} + E_{\text{ground}} \tag{4}$$

where L is the number of cruise legs and P_j and t_j are the power and duration of leg j. Leg power follows rotor momentum theory [5], with a constant term, an induced term and a parasitic term:

$$P(v) = P_0 + k_i W v_i(v) + k_p v^3 \tag{5}$$

$$v_i(v) = \sqrt{-\frac{v^2}{2} + \sqrt{\frac{v^4}{4} + v_h^4}}, v_h = \sqrt{\frac{W}{2\rho A}} \tag{6}$$

where W is the vehicle weight including payload, ρ the air density and A the rotor disc area; the four coefficients are fitted with the airframe mass constrained to the Matrice 100 range of 2.4 to 3.6 kg. Leg time is the distance divided by speed plus an acceleration and braking allowance (about 0.5 s at 4 m/s and 4.4 s at 12 m/s). Climb, descent and hover energies are modelled from altitude, payload, wind and the number of legs, and ground energy is a fixed

1.1 Wh. Since every recorded flight flies the same loop, distance effects are learned leg by leg from 587 legs of 193 usable flights.

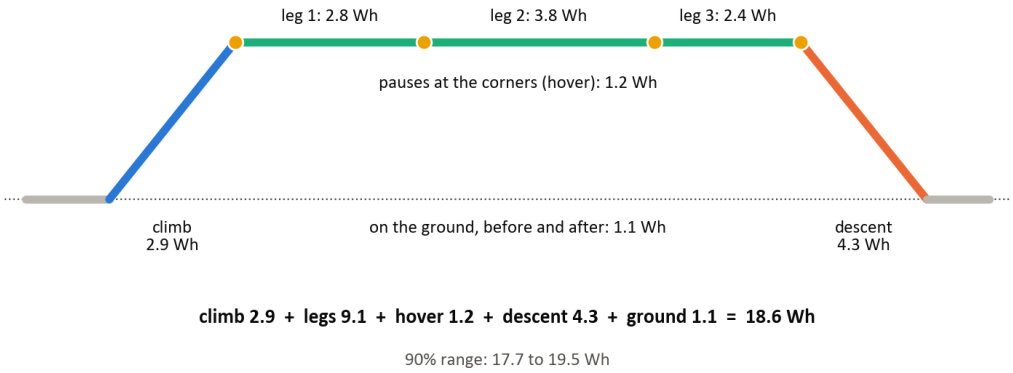


Fig. 4: Decomposition of a mission into climb, cruise legs, hover, descent and ground phases, with a worked example of 18.6 Wh

For each component, physics alone, Linear Regression, Random Forest, XGBoost and a hybrid (physics followed by XGBoost on the physics residual) were compared. The adopted *physics-first* rule uses a physics-shaped component unless a pure learning model is more than 5% better. It selected physics with XGBoost correction for leg power, leg time and descent, and physics alone for climb and hover. Sequence models are not used for Model B because a mission plan is a handful of fixed numbers, not a time series.

3.1.5 Calibrated uncertainty

A predictive range is calibrated if a nominal 90% range contains the truth about 90% of the time on unseen data. For Model A, out-of-fold predictions of every development battery are converted into standardised residuals

$$r = \frac{E - \hat{E}}{\hat{\sigma}} \tag{7}$$

where $\hat{E}$ and $\hat{\sigma}$ are the ensemble estimate and spread. Their quantiles q_β are computed with every battery given equal total weight, so that thousands of readings from one battery do not count as independent evidence, and the calibrated range for a new reading is

$$C_{1-\alpha} = [\hat{E} + q_{\alpha/2}\hat{\sigma}, \hat{E} + q_{1-\alpha/2}\hat{\sigma}] \tag{8}$$

Quantiles are stored at 23 levels between 1% and 99%. The empirical lower quantile (2.08 spreads below the estimate at the 5% level, against 1.64 for a normal distribution) shows that the model's over-estimates are heavier-tailed than a Gaussian assumes. For Model B, each held-out flight's set of component errors is replayed on the components of a new mission, and the spread of the resulting totals is the predictive distribution. The errors of one flight are kept together because they are correlated (descent and ground errors have a correlation of −0.63), and sorties flown back to back on one battery share one error set, because about 46% of their error is common.

3.1.6 Feasibility decision and evaluation design

P(success) is computed deterministically. Model A's distribution is represented by N = 400 equally likely values a_n, and for each the cumulative distribution F_B of Model B gives the probability that the mission needs no more energy than a_n:

$$P(\text{success}) = \Pr(E_{\text{avail}} \geq E_{\text{req}}) \approx \frac{1}{N}\sum_{n=1}^{N} F_B\,(a_n) \tag{9}$$

Decisions are evaluated without flying by pairing real battery states (readings with a known true remaining energy) with real missions (recorded flights, or two or three flights actually flown back to back on one battery) whose true energy is known; a pair is a success if the true energy available covers the true energy required. Twenty thousand such pairs were scored under four policies: P1, minutes left ≥ mission duration (the brief, given the true duration); P2, energy point estimates; P3, EnduroSense; and P4, an oracle. An *unsafe approval* is an approved mission that would breach the reserve, counted as a share of failing missions, and a *wasted refusal* is a refused mission that would have succeeded, counted as a share of feasible missions. A fleet simulation of four drones serving 40 tasks a day for 200 days, in which each battery follows a real recorded chain and each task is a real flight, measures the effect of each policy on a fleet.

3.1.7 Extensions: fleet planner and edge computing

Two extensions were built on top of the finished models, with real batteries and missions and simulated schedules. The **energy-aware fleet planner** serves four drones with payload limits of 750, 500, 500 and 250 g and 100 tasks in an 8-hour day, with high, normal and low priorities due within 30, 90 and 240 minutes, two 100 W chargers and battery swaps at the depot. Whenever drones are free it lists the allowed drone-task pairs (payload within limit, deadline reachable, P(success) ≥ 0.95), scores them by priority, deadline and battery fit, and matches all of them at once with the Hungarian algorithm [19]. The **edge-computing choice** decides for each task's computing job whether to process it on board (15 W), on an edge server at the depot, or on another drone waiting there, using a Shannon-capacity link model with distance loss, fading and blockage. Because half of the jobs need their result on site, the dominant cost is hovering at about 539 W (measured on development flights) while waiting; the rule picks the option with the least energy that meets the job's deadline, and P(success) before take-off includes a bad-case computing energy.

3.2 Hardware & Software Specifications

All experiments ran on a single laptop. Table 4 lists the environment; library versions are pinned in the project's requirements file so that the pipeline can be rebuilt exactly.

Component	Specification
Processor	13th Gen Intel Core i5-13420H
Memory	16 GB RAM
GPU	NVIDIA GeForce RTX 4050 Laptop GPU (CUDA 11.8), used for training the recurrent networks
Operating system	Windows 11
Language	Python 3.11
Learning libraries	PyTorch 2.7.1 [22], scikit-learn 1.4.2 [23], XGBoost 3.2.0
Data and numerics	NumPy 1.26.4, pandas 2.1.4, SciPy 1.11.4, PyArrow
Visualisation and dashboard	Matplotlib 3.8.4, Streamlit 1.64.0, Altair 6.3.0
Testing	pytest 9.1.1 (165 automated tests)

Table 4: Hardware and software environment

Inference cost is small: on a laptop CPU with one thread every model predicts one reading in 6 ms or less (the GRU in about 4.8 ms with a 0.18 MB model), which is compatible with an on-board companion computer. A full rebuild of all results from the raw data takes about 30 minutes with one command.

3.3 Dataset Description

The work uses the CMU dataset "Data Collected with Package Delivery Quadcopter Drone" [1], chosen because it is real flight data, records voltage and current so that energy can be computed exactly, and varies speed, payload and altitude in a planned grid. Its main facts are given in Table 5.

Property	Value
Vehicle	DJI Matrice 100 quadcopter
Battery	DJI TB47D, 6 cells, 4.5 Ah × 22.2 V = 99.9 Wh
Flights	209 over 18 flying days (April to October 2019)
Sensor readings	257,896 at about 8 Hz
Signals	Battery voltage and current, on-board anemometer wind, GPS position, velocity, orientation
Experimental grid	Speeds 4, 6, 8, 10, 12 m/s × payloads 0, 250, 500 g × altitudes 25, 50, 75, 100 m
Routes	R1 (closed loop of 140, 198 and 119 m legs) for 182 flights; R2 to R7 and hover tests for the rest
Typical flight	About 3 minutes and 21 Wh
Derived for this work	90 battery chains; 60 labelled for Model A (29,592 one-second rows); 587 legs for Model B

Table 5: Characteristics of the CMU package-delivery drone dataset

Several problems in the raw data were found and corrected in code, without editing the raw files: flight start times stored as text sorted 7 days out of order and had to be ordered as timestamps; 4 start times were mis-logged and were corrected with voltage evidence; the sign of vertical velocity is opposite to the documentation (its correlation with the change in altitude is +0.77); and the altitude reading drifts by up to 14 m after landing, so motor state is

inferred from current instead. Batteries that start full deliver 64.7 ± 7.3 Wh before the reserve, and it is this spread between batteries that Model A has to learn.

Chapter 4
Results and Discussion

4.1 Evaluation Protocol

All development results come from five-fold cross-validation grouped by battery chain. The locked test set was opened on 4 October 2026, after a rehearsal of the evaluation code on development data and a from-scratch rebuild that reproduced every stored result. Eleven of the 16 test chains carry an energy label and are used for Model A (5,708 readings); all 40 flights are used for Model B, seven of them on routes absent from the development data. Every opening of the test set is logged with a fingerprint of the code and settings. Differences between models are reported with 95% intervals from a bootstrap that resamples whole batteries 2,000 times [24]. Table 6 summarises the outcome of the four pre-registered criteria.

Criterion (fixed before testing)	Result	Evidence
C1. Model A beats both non-learning baselines	Met	2.19 Wh against 2.90 Wh (voltage lookup) and 4.21 Wh (energy counting)
C2. Model B within 5%, including unseen routes	Not met	3.14% on all test flights; 5.66% on unseen routes
C3. 90% ranges cover 88–92%	Not met, within sampling noise	Model A 93.8% (plausible 83.0–100%); Model B 80.0% (65.0–92.5%)
C4. Better decisions than minutes-left and point estimates	Met	Unsafe approvals 0.61% against 12.36% and 3.58%; AUC 0.994

Table 6: Pre-registered success criteria and their outcome on the locked test set

4.2 Energy Available (Model A)

Table 7 compares all models on readings with a pre-flight rest voltage, in cross-validation and on the test set, and Figure 5 plots one against the other.

Model	CV (Wh)	Test (Wh)	Test vs. voltage lookup
GRU ensemble, calibrated (final)	2.37	2.19	−0.71 (−1.33 to −0.12)
LSTM	2.45	2.32	−0.58 (−1.18 to +0.06)
XGBoost + physics	2.59	2.40	−0.49 (−1.02 to +0.01)
Linear Regression	2.64	2.63	−0.27 (−0.92 to +0.41)
Random Forest	2.72	2.71	−0.19 (−0.85 to +0.58)
XGBoost	2.91	2.75	−0.14 (−0.56 to +0.30)
GRU (single)	2.39	2.81	−0.08 (−0.73 to +0.62)
Voltage lookup (no ML)	2.82	2.90	—
Energy counting (no ML)	3.98	4.21	+1.31 (+0.60 to +1.93)

Table 7: Model A mean absolute error (Wh) on comparable readings; the last column is the test difference from the voltage-lookup baseline with its 95% interval

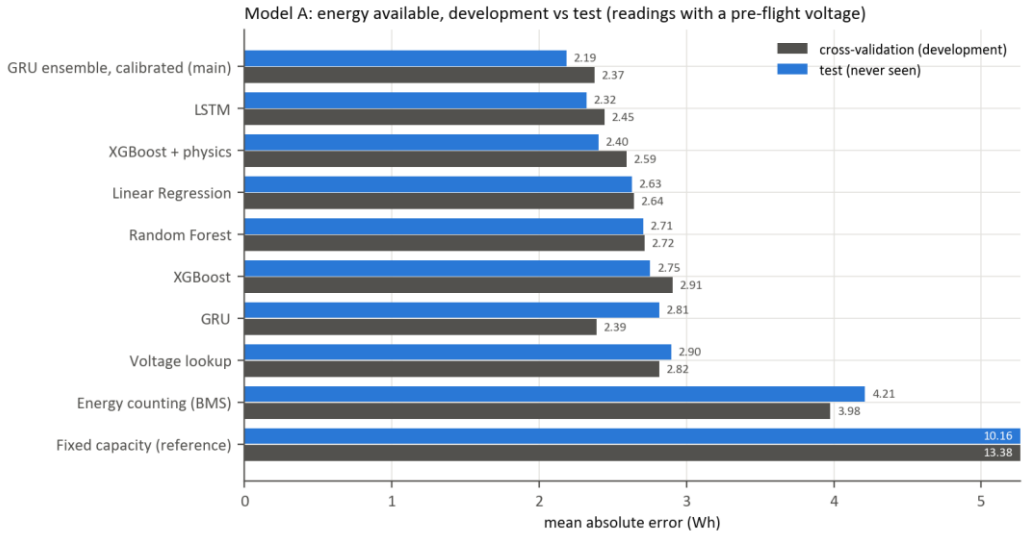


Fig. 5: Model A: cross-validation error against test error for each model

The calibrated GRU ensemble held up on unseen batteries with 2.19 Wh, and it is the only model whose advantage over the voltage-lookup baseline is clear on test, being better on 9 of the 11 batteries. The recurrent models benefit from observing how voltage responds to load over the last minute, which reflects the condition of the individual battery. A single GRU, the best model in cross-validation (2.39 Wh), scored 2.81 Wh on test, no better than the baseline: averaging five networks is what made the result dependable. Tree models gained little over the baseline, because with only a few dozen development batteries they latch onto battery-specific patterns that a battery-grouped split exposes and a random row split would have hidden.

4.3 Energy Required (Model B)

Flights	n	MAPE	MAE (Wh)
Cross-validation (development)	153	2.35%	0.49
All test flights	40	3.14%	0.70
Route flown in training (R1)	33	2.61%	0.56
Unseen routes (R2, R3, R6)	7	5.66%	1.35
of which R6, the longest	5	5.86%	1.48

Table 8: Model B mission-energy error by group of flights

As Table 8 shows, on the trained route Model B generalises to new batteries and days with 2.6% error. On unseen routes it misses the 5% target with 5.7%, but in the safe direction: all seven flights were over-predicted, with the excess in the cruise legs of R6, whose cruise distance (about 820 m) is roughly 45% longer than any route in development. In development, holding out a whole altitude or payload confirmed the physics-first choice: tree models reached 14% and 9–10% error respectively, while the physics-first model stayed at 2.5% and 3.1%. Wind added little, because only wind speed is known and head and tail winds cancel on a closed loop.

4.4 Calibration of the Predictive Ranges

In development, Model A's 90% ranges covered 89.5% of held-out readings with a mean width of 11.2 Wh, and Model B's covered 91.5% with a width of 2.1 Wh. On test (Figure 6), Model A covered 93.8%; almost all misses came from two batteries whose capacity was over-estimated (by 7.4 and 5.1 Wh on average), while the other nine batteries were inside their range for at least 99.8% of their readings. Model A's error is thus mainly a fixed per-battery offset (about 3.3 Wh between batteries against 1.6 Wh within one), so risk is per battery rather than per moment. Model B's ranges were too narrow on test at 80.0%; four of the six misses on the trained route are at 100 m altitude, a weak spot already flagged during development. Both coverages lie inside the intervals expected from so few batteries and flights, which is why criterion C3 is reported as not met but within sampling noise.

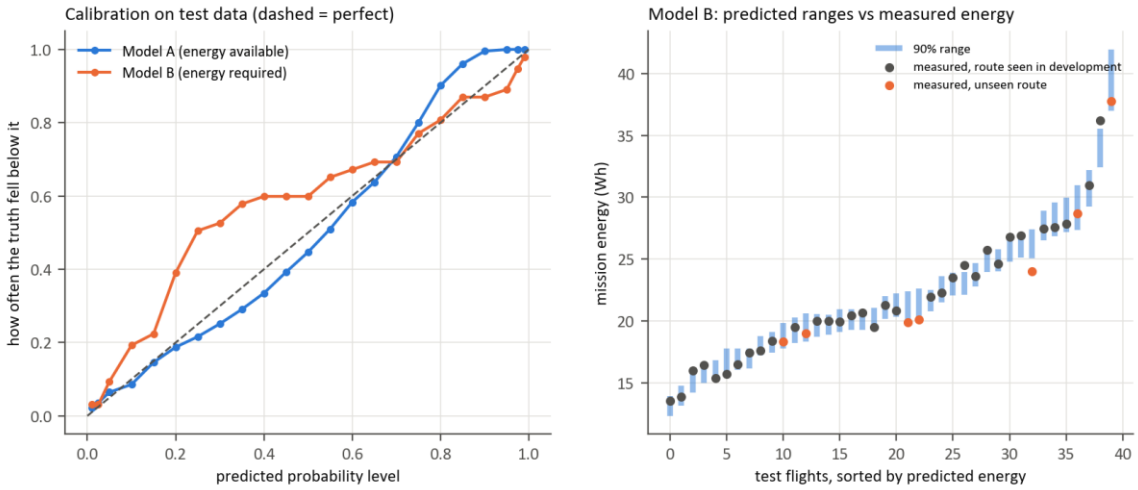


Fig. 6: Left: calibration of both models on test data (the dashed line is perfect calibration). Right: Model B 90% ranges against measured mission energy

4.5 Feasibility Decisions

Rule	Unsafe, dev.	Unsafe, test	Wasted refusals, test	Unsafe at take-off, test
P1: minutes left ≥ duration	10.5%	12.4%	3.3%	12.8%
P2: energy point estimates	2.9%	3.6%	3.9%	4.3%
P2 + margin tuned in development	—	0.55%	24.4%	0.00%
P3: EnduroSense, $\tau = 0.95$	0.25%	0.61%	24.3%	0.00%

Table 9: Decision quality of each rule. Unsafe approvals are a share of missions that would breach the reserve; wasted refusals are a share of feasible missions

The headline result holds on unseen data (Table 9). The minutes-left rule approves about one in eight missions that would fail; EnduroSense approves about one in 160, and none at take-off decisions. All of EnduroSense's unsafe approvals involve the same two batteries that Model A over-estimated. The price is caution: 24% of feasible missions are refused, but these are close calls with a median true margin of only 3.8 Wh. The minutes-left rule cannot be rescued by a safety margin, since the margin that made it as safe in development refuses 91% of feasible missions. Point estimates with a well-tuned fixed margin performed similarly to

P(success) over all cases; at take-off decisions the probability refused fewer feasible missions (25% against 32%) at the same safety, and it required no tuning. Missions rated at 95–99% succeeded 94.2% of the time, which is slightly over-confident for the same per-battery reason. Figure 7 shows the full trade-off as each rule's threshold varies.

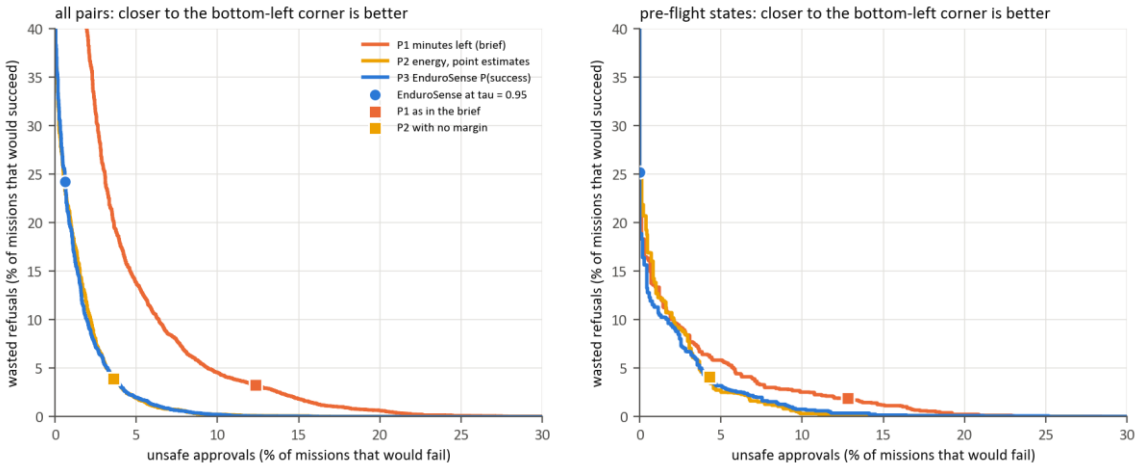


Fig. 7: Unsafe approvals against wasted refusals as each rule's threshold moves, on test data; closer to the bottom-left corner is better

4.6 Fleet Simulation

Rule	Unsafe per 100 flown	Completed per day (of 40)	Battery swaps per day
Minutes left (the brief)	12.0	40.0	15.5
Energy point estimates	4.9	40.0	16.2
Point estimates + development margin	0.43	39.6	18.4
EnduroSense, $\tau = 0.95$	0.73	39.1	17.0
Oracle	0.00	39.9	16.2

Table 10: Fleet simulation on eight unseen batteries (four drones, 40 tasks per day, 200 days)

In the fleet (Table 10 and Figure 8), EnduroSense produces sixteen times fewer unsafe missions than the brief's rule for about 10% more battery swaps, while completing almost every task. Against point estimates with a margin it sits at a nearby point of the same trade-off, with slightly more unsafe missions and fewer swaps.

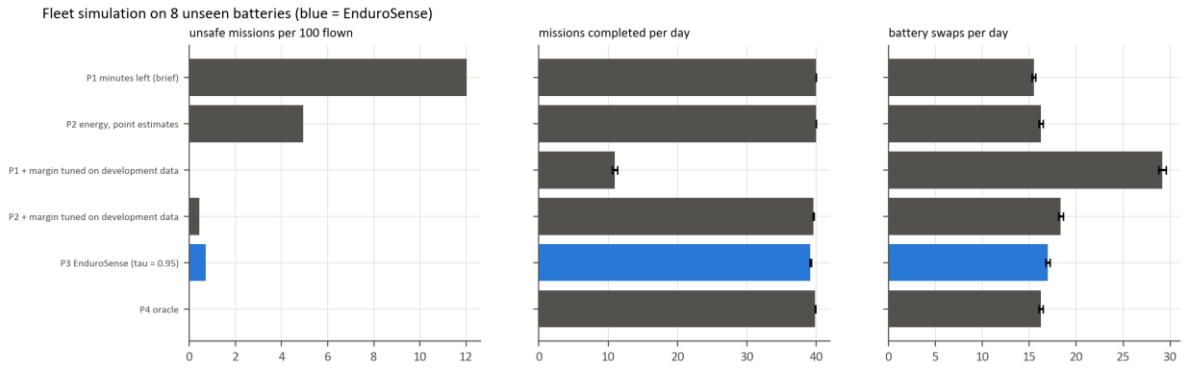


Fig. 8: Fleet simulation on unseen batteries: unsafe missions, completed tasks and battery swaps per rule

4.7 Extensions: Fleet Planner and Edge Computing

Both extensions were developed on development batteries, their three criteria were written down beforehand, and a single logged run was made on the eight unseen batteries; no model or earlier result changed. Tables 11 and 12 and Figure 9 report the outcome.

Policy	Weighted on time	High priority on time	Unsafe per 100	Swaps per day
First come, minutes left (the brief)	67.2%	53%	11.10	31.7
First come, EnduroSense check	71.3%	62%	0.62	34.4
Planner, best estimates	84.3%	96%	4.46	33.8
Planner, EnduroSense	83.8%	95%	0.61	37.2
Planner, oracle	84.3%	96%	0.00	34.0

Table 11: Energy-aware fleet planner on unseen batteries (simulated days)

Rule	Computing energy per task (Wh)	Jobs on time	On board / edge / drone (%)
Always on board	2.60	90.2%	100 / 0 / 0
Always offload	0.95	81.5%	0 / 100 / 0
Energy-aware	2.05	95.6%	37 / 62 / 1

Table 12: Where to process each task's data, with the EnduroSense planner

Planning is what gets urgent work done: 95% of high-priority tasks finish on time against 62% for a first-come queue. The EnduroSense check is what keeps it safe: the planner with best estimates is equally productive but has 4.46 unsafe missions per 100 against 0.61. The energy-aware computing rule meets more job deadlines than either fixed rule (95.6%) for less energy than processing on board. “Always offload” appears cheap only because the planner refuses many on-site jobs before take-off, since a weak link could mean minutes of hovering; as a result only 56.7% of priority-weighted work is done on time. All three extension criteria were met. The computing settings are typical published values, not measurements, and these results are simulations on real batteries and missions.

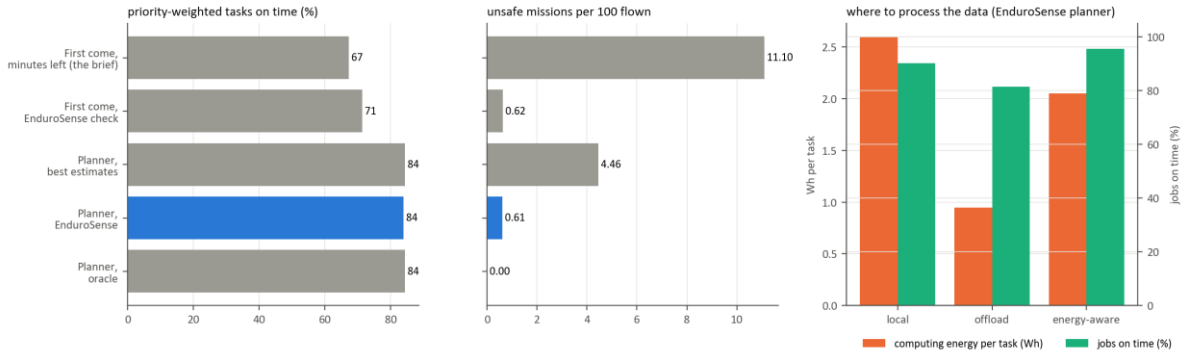


Fig. 9: The two extensions on unseen batteries (simulated schedules)

4.8 Discussion and Limitations

The test set supports four claims: a calibrated GRU ensemble estimates energy to reserve to about 2.2 Wh on unseen batteries and beats a fair non-learning baseline; mission energy is predicted to about 3% for missions like those flown in training; deciding on energy with a calibrated margin cuts unsafe approvals from about 12% to 0.6%, and to zero at take-off; and the remaining unsafe cases trace to the per-battery risk identified during development. It does not support claims that Model B is within 5% on routes much longer than its training routes, that Model B's ranges are calibrated on new data, that a probability is measurably better than point estimates with a well-chosen margin, or that a single GRU beats the baseline. These weaknesses were reported rather than patched, because fixing them and scoring again on the same test set would make the results optimistic.

The main limitations are the small test set (11 labelled batteries and 40 flights from one drone and one battery type), the uncertainty of the label itself (about 1.5–2 Wh), decisions evaluated by pairing recorded states and missions rather than by new flights, and conditions outside the data, such as other airframes, temperatures, battery ages and stronger winds, which remain untested. Model B warns when a mission falls outside the ranges it was tested on.

Chapter 5
Conclusion and Future Scope

This project replaced the remaining-flight-time formulation of UAV mission planning with an energy-based, uncertainty-aware one. EnduroSense predicts the energy a battery can still deliver above a safe reserve and the energy a specific mission will require, calibrates both predictions on unseen batteries and flights, and combines them into a probability of success that drives the go/no-go decision and a fleet scheduler. Evaluated once on a locked test set with criteria fixed in advance, the system reduced the error of battery energy estimation from 2.90 Wh to 2.19 Wh, predicted mission energy to 3.1%, and reduced unsafe approvals from 12.4% under the minutes-left rule to 0.61%, with none at take-off decisions. Two of the four criteria were met in full; the other two, mission accuracy on much longer routes and the calibration of the mission ranges, fell short and are reported as such. The most robust gain comes from deciding on energy with a calibrated safety margin instead of on minutes.

The work also demonstrates practices that make such results trustworthy: battery-grouped evaluation, a gated and logged test set, pre-registered criteria, a one-command byte-identical rebuild, 165 automated tests including mutation testing, and six adversarial verification passes. The future scope of the work includes:

- **More batteries and routes.** A second battery type, longer routes and more flights at 100 m altitude would address the two missed criteria directly.
- **Per-battery learning.** Because Model A's error is mainly a per-battery offset, updating each battery's capacity estimate after every flight should narrow the ranges and remove the dominant source of unsafe approvals.
- **Range widening outside the training envelope.** Using route length as an input to Model B's range would widen it for missions longer than those seen in training.
- **On-board deployment.** Timing the models on a companion computer such as a Raspberry Pi, and integrating the decision with a flight controller for in-flight re-planning.
- **Richer operating conditions.** Wind direction along the route, temperature and battery ageing, and measured rather than assumed radio and edge-server characteristics for the computing extension.

References

[1] T. A. Rodrigues, J. Patrikar, A. Choudhry, J. Feldgoise, V. Arcot, A. Gahlaut, S. Lau, B. Moon, B. Wagner, H. S. Matthews, S. Scherer, and C. Samaras, "In-flight positional and energy use data set of a DJI Matrice 100 quadcopter for small package delivery," *Scientific Data*, vol. 8, Art. no. 155, 2021. Dataset: https://doi.org/10.1184/R1/12683453

[2] J. Zhang, J. F. Campbell, D. C. Sweeney II, and A. C. Hupman, "Energy consumption models for delivery drones: A comparison and assessment," *Transportation Research Part D: Transport and Environment*, vol. 90, Art. no. 102668, 2021.

[3] H. V. Abeywickrama, B. A. Jayawickrama, Y. He, and E. Dutkiewicz, "Comprehensive energy consumption model for unmanned aerial vehicles, based on empirical studies of battery performance," *IEEE Access*, vol. 6, pp. 58383–58394, 2018.

[4] T. A. Rodrigues, J. Patrikar, N. L. Oliveira, H. S. Matthews, S. Scherer, and C. Samaras, "Drone flight data reveal energy and greenhouse gas emissions savings for very small package delivery," *Patterns*, vol. 3, no. 8, Art. no. 100569, 2022.

[5] J. G. Leishman, *Principles of Helicopter Aerodynamics*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2006.

[6] G. L. Plett, *Battery Management Systems, Volume I: Battery Modeling*. Norwood, MA, USA: Artech House, 2015.

[7] B. Saha, E. Koshimoto, C. C. Quach, E. F. Hogge, T. H. Strom, B. L. Hill, S. L. Vazquez, and K. Goebel, "Battery health management system for electric UAVs," in *Proc. IEEE Aerospace Conference*, 2011, pp. 1–9.

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[12] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.

[13] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 6402–6413.

[14] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.

[15] G. Shafer and V. Vovk, "A tutorial on conformal prediction," *Journal of Machine Learning Research*, vol. 9, pp. 371–421, 2008.

[16] Y. Romano, E. Patterson, and E. Candès, "Conformalized quantile regression," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 3543–3553.

[17] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," arXiv:2107.07511, 2021.

[18] K. Dorling, J. Heinrichs, G. G. Messier, and S. Magierowski, "Vehicle routing problems for drone delivery," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 47, no. 1, pp. 70–85, 2017.

[19] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[20] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[21] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[22] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[23] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[24] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.